



机器视觉实验 2

姓名：冯怡然

学号：2411434

班级：人工智能特色班

日期：2026 年 3 月 16 日

南开大学 · 机器视觉实验

1 题目一：数字图像的度量和拓扑性质——距离变换

1.1 实验目的

1. 理解数字图像中距离度量（如曼哈顿距离、欧几里得距离）的基本概念。
2. 掌握两遍扫描算法（Two-pass Algorithm）实现距离变换的基本原理。
3. 能够利用 Matlab 编写程序，实现对二值图像子集 S 的斜切（距离变换）。

1.2 实验原理

距离变换（Distance Transform）是将二值图像转换为灰度图像的过程。图像中的每个像素值代表其到最近目标像素（子集 S ）的距离。本次实验采用基于动态规划的两遍扫描算法：

- **初始化**：创建一个与原图大小相同的数组 F 。目标集合 S 内的元素位置置为 0，其余位置置为无穷大 (∞)。
- **前向扫描 (AL 集合)**：从上到下、从左到右遍历。利用当前像素左侧和上方的邻接像素更新当前像素值：

$$F(p) = \min_{q \in AL} [F(p), D(p, q) + F(q)]$$

- **后向扫描 (BR 集合)**：从下到上、从右到左遍历。利用当前像素右侧和下方的邻接像素更新当前像素值：

$$F(p) = \min_{q \in BR} [F(p), D(p, q) + F(q)]$$

1.3 实验步骤

1. **图像预处理**：读入彩色或灰度图像，通过 `rgb2gray` 转换为灰度图，并使用 `imbinarize` 转换为二值图像。
2. **定义目标集**：根据实验要求（如黑块为物体），对二值图像进行必要的取反，确保目标子集 S 的像素值为 0。
3. **两遍扫描实现**：
 - 编写循环结构，首先执行由左上至右下的扫描，处理 AL 邻域。
 - 随后执行由右下至左上的扫描，处理 BR 邻域。
4. **结果可视化**：利用 `imshow(F)` 展示图片

1.4 程序代码 (Matlab)

1.4.1 MyDisTrans.m

```
1 function DisTrans_result=MyDisTrans(bw)
2 [M, N] = size(bw);    %完善距离变换的计算过程
3 F = inf(M, N);
4 F(bw == 0) = 0;
5     for i = 1:M        % 第一遍扫描：左上 -> 右下
6         for j = 1:N
7             if i > 1
8                 F(i,j) = min(F(i,j), 1 + F(i-1, j));
9             end
10            if j > 1
11                F(i,j) = min(F(i,j), 1 + F(i, j-1));
12            end
13        end
14    end
15    for i = M:-1:1      % 第二遍扫描：右下 -> 左上
16        for j = N:-1:1
17            if i < M
18                F(i,j) = min(F(i,j), 1 + F(i+1, j));
19            end
20            if j < N
21                F(i,j) = min(F(i,j), 1 + F(i, j+1));
22            end
23        end
24    end
25 DisTrans_result=F;    %返回原二值图像经过距离变换后的结果DisTrans_result
26 end
```

Listing 1: MyDisTrans.m

1.4.2 main.m

```
1 clc;clear all;close all;
2 img = imread('ratatouille.png');    % 读入图像
3 img_gray = rgb2gray(img);           % 将读入的彩色图像转换为二值图像
4 bw = imbinarize(img_gray);
5 DisTrans_result = MyDisTrans(bw);   %距离变换，需编程实现
6 figure;
7 imshow(DisTrans_result);
8 title('DisTrans-result');
```

Listing 2: main.m

1.5 实验结果显示

原图片如下图所示



图 1: 原图片

运行结果如图所示:

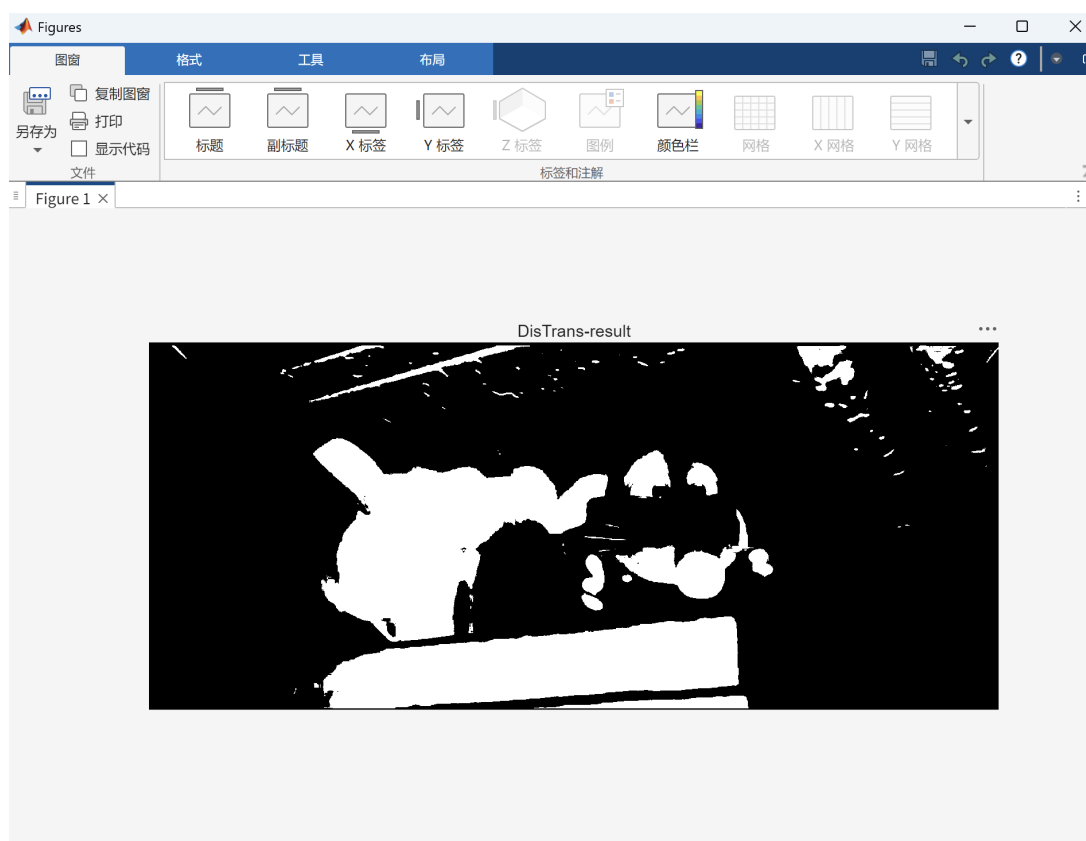


图 2: 处理图片

1.6 实验分析总结

通过本次对图像距离变换（Distance Transform）算法的编程实现与调试，得出以下分析结论：

1. **图像维度对算法的影响**：在调试过程中发现，直接对彩色图像（RGB）进行处理会导致 `size` 函数返回三元组，从而引发宽度 N 变为原始的 3 倍。实验证明，在进行距离变换前，必须执行 `rgb2gray` 预处理。这反映了数字图像处理中“单通道灰度化”是进行拓扑性质分析的基础前提。而如果没有进行灰度预处理，就会返回三个通道的图片（如下图）。



图 3: 三只 Remy!

2. **两遍扫描法（Two-pass）的优越性**：该算法利用动态规划思想，将全局搜索简化为局部邻域计算。

- 前向扫描传播了左上方的信息；
- 后向扫描传播了右下方的信息。

两遍扫描后，每个像素均能获得到最近边界的最短曼哈顿距离。相比于暴力搜索每个像素点的 $O((MN)^2)$ 复杂度，该算法 $O(MN)$ 的线性复杂度极大地提升了处理大尺寸图像的效率。

3. **数值映射**：通过使用 `colormap(jet)` 等伪彩色映射，可以直观地观察到距离场的梯度分布（越靠近物体中心，颜色越趋向红色）。

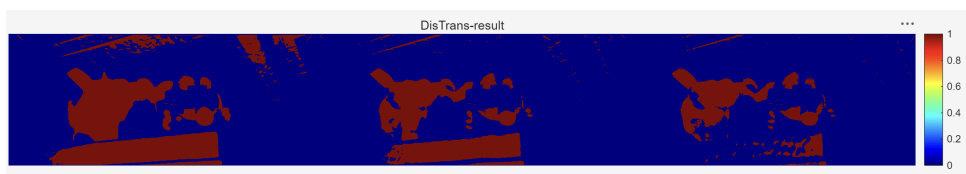


图 4: 三只彩色 Remy!

结论：本算法成功实现了从二值逻辑信息到空间几何距离信息的转换。

2 题目四——计算积分图像

2.1 实验目的

1. 深入理解数字图像中“积分图像” (Summed-area table) 的基本概念及其在图像处理中的重要性。
2. 掌握积分图像的高效递归构建算法。
3. 学习并运用 Matlab 编程语言实现矩阵的累加运算，提升对二维矩阵数据的处理能力。
4. 验证积分图像在快速计算图像子区域像素和方面的优越性。

2.2 实验原理

积分图像 $ii(i, j)$ 是原图像 $f(i, j)$ 的一种表示形式。其定义为图像中位于像素点 (i, j) 左上角所有像素的和：

$$ii(i, j) = \sum_{x \leq i, y \leq j} f(x, y) \quad (1)$$

为了实现高效计算，通常采用一次遍历图像的递推算法。通过引入行方向的累加和 $s(i, j)$ ，可以避免重复计算：

- 行累加和递推： $s(i, j) = s(i, j - 1) + f(i, j)$
- 积分图像值递推： $ii(i, j) = ii(i - 1, j) + s(i, j)$

其中，初始边界条件通常设为： $s(i, -1) = 0$ 且 $ii(-1, j) = 0$ 。

2.3 实验步骤 (Matlab)

1. 初始化矩阵：

- 获取原图像 f 的尺寸 $[M, N]$ 。
- 创建一个同样大小（或大小为 $(M + 1) \times (N + 1)$ 以方便边界处理）的全零矩阵作为积分图像 ii 。
- 初始化行方向累加中间变量 s 。

2. 双层循环递归计算：按照逐行扫描的顺序，使用嵌套循环遍历图像：

- 对于当前像素 (i, j) ，根据公式计算当前行的累加值 $s(i, j)$ 。
- 结合上一行相同位置的积分值 $ii(i - 1, j)$ ，更新当前位置的积分图像值 $ii(i, j)$ 。

3. 结果校验与可视化：

- 将手写算法的结果与 Matlab 自带函数（如 `integralImage`）进行对比，验证准确性。
- 使用 `imagesc` 命令显示生成的积分图像。

2.4 程序代码 (Matlab)

2.4.1 MyDisTrans.m

函数中采取了反向积分

```
1 function MyInteImg_result=MyInteImg(img)
2 %完善积分图像的计算过程
3 [rows,cols]=size(img);
4 MyInteImg_result = zeros(rows, cols);
5 s = zeros(rows, cols);
6 for i=rows:-1:1
7     for j=cols:-1:1
8         if j==cols
9             s(i,j)=img(i,j);
10        else
11            s(i,j)=s(i,j+1)+img(i,j);
12        end
13        if i==rows
14            MyInteImg_result(i,j)=s(i,j);
15        else
16            MyInteImg_result(i,j)=MyInteImg_result(i+1,j)+s(i,j);
17        end
18    end
19 end
20 %返回原图像计算得到的积分图像MyInteImg_result
21 end
```

Listing 3: MyDisTrans.m

2.4.2 main.m

```
1 clc;clear all;close all;
2 % 读入图像
3 img = imread('ratatouille.png');
4 % 将读入的彩色图像转换为灰度图像
5 img = im2gray(img);
6 img=double(img);
7 %计算积分图像，需编程实现
8 InteImg_result = MyInteImg(img);
9 figure;
10 imshow(InteImg_result,[]);
11 title('InteImg-result');
12 %测试对照
13 img0=integralImage(img);
14 figure;
15 imshow(img0,[]);
16 title('system-result');
```

Listing 4: main.m

2.5 实验结果显示

原图片如下图所示



图 5: 原图片

运行结果如图所示:



图 6: 处理图片

2.6 实验分析总结

2.6.1 1. 算法实现与性能分析

本实验通过 Matlab 编程实现了积分图像 (Integral Image) 的快速构建。该算法的核心在于利用递归思想将二维累加复杂度降低至 $O(MN)$ 。

- **递归有效性:** 通过引入行累加中间变量 $s(i, j)$, 我们能够仅凭当前像素值和已计算出的相邻单元格 (左侧与上方) 完成更新, 避免了冗余计算。
- **动态规划特质:** 积分图像本质上是一种空间域的动态规划, 每一格都存储了从原点到该点矩形区域的能量总和。

2. 关于“逆向积分”与中心对称的发现

在实验过程中, 通过修改循环方向 (如使用 `rows:-1:1` 步长) 实现了逆向积分计算。

- **对称性原理:** 传统积分图像是从左上 $(1, 1)$ 向右下 (M, N) 累加。若采用逆向扫描 (从右下向左上累加), 生成的图像在数值分布上与原积分图像互为“镜像”。
- **结论:** 这种逆向积分实际上是改变了积分的参考原点。由于积分算子在离散空间上的线性特质, 当我们将起始点从左上角移至右下角时, 所生成的图像呈现出明显的 * 中心对称 * 特征。这一特性可以用于某些特定场景下的双向特征提取或对称性检测。

3. 调试经验与技术总结

针对实验过程中出现的异常现象, 总结解决方案如下:

1. **数值溢出处理:** 原始图像通常为 `uint8` 类型, 其最大值为 255。而在积分过程中, 像素值累加极易超过该界限。实验表明, 必须先将数据类型转换为 `double` 进行计算, 否则会导致计算截断, 产生错误的图像结果。
2. **可视化映射问题 (解决“一片空白”):** 积分图像由于右下角数值巨大, 直接使用 `imshow` 会导致所有像素显示为过饱和的白色。通过使用 `imshow(I, [])` 自动拉伸显示范围, 或使用 `imagesc` 伪彩色映射, 可以清晰地观察到能量分布的梯度变化。
3. **循环执行逻辑 (解决“一片漆黑”):** 在 Matlab 中, `for i = rows:1` 默认不会执行任何循环。必须通过指定负步长 `rows:-1:1` 确保循环能够逆向运行。此外, 必须严格检查边界索引, 防止出现下标为 0 的越界错误。

总结: 通过本次实验, 我不仅在编程层面上掌握了积分图像的构建, 更从数学对称性的角度深入理解了空间累加算子的性质。